知乎 首发于 好好学习 切换模式 写文章

# Python os 模块详解

轩辕御龙
ZGNB

324 人赞同了该文章

## Python os 模块详解

### 1. 简介

os 就是“operating system”的缩写，顾名思义， os 模块提供的就是各种 Python 程序与操作系统进行交互的接口。通过使用 os 模块，一方面可以方便地与操作系统进行交互，另一方面页可以极大增强代码的可移植性。如果该模块中相关功能出错，会抛出 OSError 异常或其子类异常。

> 注意，如果是读写文件的话，建议使用内置函数 open() ；如果是路径相关的操作，建议使用 os 的子模块 os.path ；如果要逐行读取多个文件，建议使用 fileinput 模块；要创建临时文件或路径，建议使用 tempfile 模块；要进行更高级的文件和路径操作则应当使用 shutil 模块。

当然，使用 os 模块可以写出操作系统无关的代码并不意味着 os 无法调用一些特定系统的扩展功能，但要切记一点：一旦这样做就会极大**损害代码的可移植性**。

此外，导入 os 模块时还要小心一点，千万**不要**为了图调用省事儿而将 os 模块解包导入，即不要使用 from os import * 来导入 os 模块；否则 os.open() 将会覆盖内置函数 open() ，从而造成预料之外的错误。

### 2. 常用功能

> 注意， os 模块中大多数接受路径作为参数的函数也可以接受“文件描述符”作为参数。
> 文件描述符：file descriptor，在 Python 文档中简记为 fd，是一个与某个打开的文件对象绑定的整数，可以理解为该文件在系统中的编号。

#### 2.1 os.name

该属性宽泛地指明了当前 Python 运行所在的环境，实际上是导入的操作系统相关模块的名称。这个名称也决定了模块中**哪些功能是可用**的，哪些是没有相应实现的。

目前有效名称为以下三个： posix ， nt ， java 。

其中 posix 是 Portable Operating System Interface of UNIX（可移植操作系统接口）的缩写。Linux 和 Mac OS 均会返回该值； nt 全称应为“Microsoft Windows NT”，大体可以等同于 Windows 操作系统，因此 Windows 环境下会返回该值； java 则是 Java 虚拟机环境下的返回值。

因此在我的电脑（win10）上执行下述代码，返回值是 nt ：

```
>>> import os
>>> os.name
'nt'
```

而在 WSL（Windows Subsystem Linux，Windows 下的 Linux 子系统）上的结果则是：

```
>>> import os
>>> os.name
'posix'
```

▲ 赞同 324 ▼ 14 条评论 分享 喜欢 收藏 申请转载 ···

> 查看 `sys` 模块中的 `sys.platform` 属性可以得到关于运行平台更详细的信息，在此不再赘述

### 2.2 os.environ

`os.environ` 属性可以返回环境相关的信息，主要是各类环境变量。返回值是一个映射（类似字典类型），具体的值为第一次导入 `os` 模块时的快照；其中的各个键值对，键是环境变量名，值则是环境变量对应的值。在第一次导入 `os` 模块之后，除非直接修改 `os.environ` 的值，否则该属性的值不再发生变化。

比如其中键为“HOMEPATH”（Windows 下，Linux 下为“HOME”）的项，对应的值就是用户主目录的路径。Windows 下，其值为：

```
>>> os.environ["HOMEPATH"]
'd:\\justdopython'
```

Linux 下，其值为：

```
>>> os.environ["HOME"]
'/home/justdopython'
```

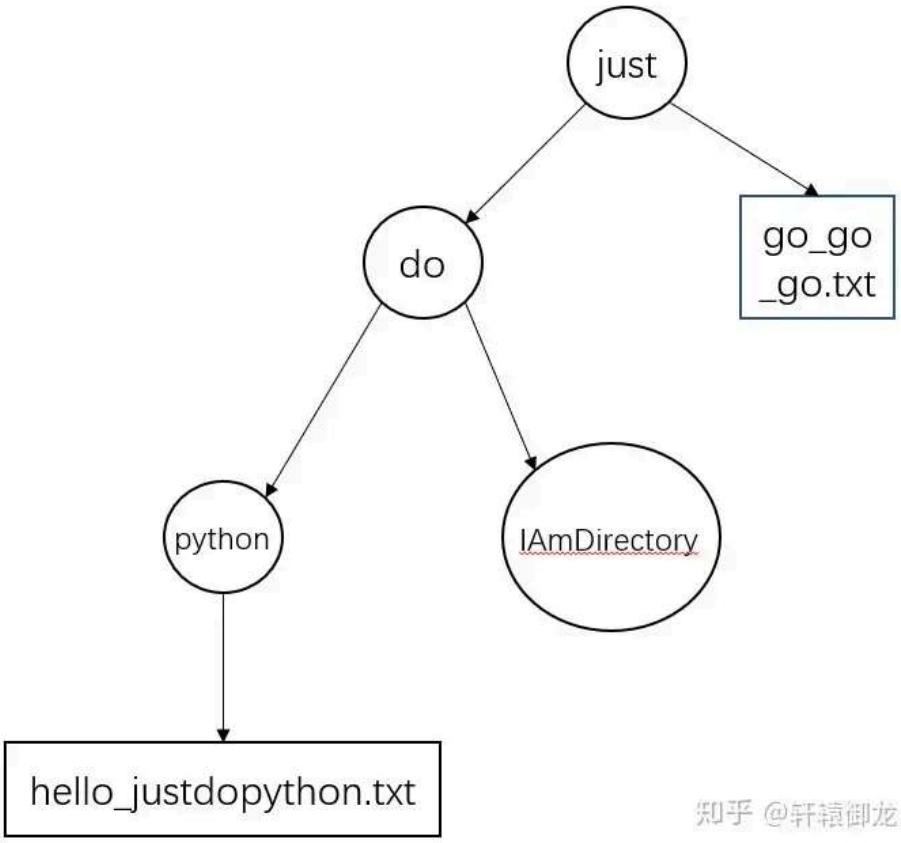
### 2.3 os.walk()

这个函数需要传入一个路径作为 `top` 参数，函数的作用是在以 `top` 为根节点的目录树中游走，对树中的每个目录生成一个由 `(dirpath, dirnames, filenames)` 三项组成的三元组。

其中， `dirpath` 是一个指示这个目录路径的字符串， `dirnames` 是一个 `dirpath` 下子目录名（除去“.”和“..”）组成的列表， `filenames` 则是由 `dirpath` 下所有非目录的文件名组成的列表。要注意的是，这些名称并不包含所在路径本身，要获取 `dirpath` 下某个文件或路径从 `top` 目录开始的完整路径，需要使用 `os.path.join(dirpath, name)` 。

注意最终返回的结果是一个迭代器，我们可以使用 `for` 语句逐个取得迭代器的每一项：

```
>>> for item in os.walk("."):
...     print(item)
...
('.', ['do'], ['go_go_go.txt'])
('.\\do', ['IAmDirectory', 'python'], [])
('.\\do\\IAmDirectory', [], [])
('.\\do\\python', [], ['hello_justdopython.txt'])
```




### 2.4 os.listdir()

“listdir”即“list directories”，列出（当前）目录下的全部路径（及文件）。该函数存在一个参数，用以指定要列出子目录的路径，默认为“.”，即“当前路径”。

函数返回值是一个列表，其中各元素均为字符串，分别是各路径名和文件名。

通常在需要遍历某个文件夹中文件的场景下极为实用。

比如定义以下函数：

```
def get_filelists(file_dir='.'):
    list_directory = os.listdir(file_dir)
    filelists = []
    for directory in list_directory:
        # os.path 模块稍后会讲到
        if(os.path.isfile(directory)):
            filelists.append(directory)
    return filelists
```

该函数的返回值就是当前目录下所有文件而非文件夹的名称列表。

### 2.5 os.mkdir()

“mkdir”，即“make directory”，用处是“新建一个路径”。需要传入一个类路径参数用以指定新建路径的位置和名称，如果指定路径已存在，则会抛出 `FileExistsError` 异常。

该函数只能在已有的路径下新建一级路径，否则（即新建多级路径）会抛出 `FileNotFoundError` 异常。

相应地，在需要新建多级路径的场景下，可以使用 `os.makedirs()` 来完成任务。函数 `os.makedirs()` 执行的是递归创建，若有必要，会分别新建指定路径经过的中间路径，直到最后

示例如下：

```
>>> os.mkdir("test_os_mkdir")
>>> os.mkdir("test_os_mkdir")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileExistsError: [WinError 183] 当文件已存在时，无法创建该文件。: 'test_os_mkdir'
>>>
>>> os.mkdir("test_os_mkdir/test_os_makedirs/just/do/python/hello")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [WinError 3] 系统找不到指定的路径。: 'test_os_mkdir/test_os_makedirs/
>>>
>>> os.makedirs("test_os_mkdir/test_os_makedirs/just/do/python/hello")
```

### 2.6 os.remove()

用于删除文件，如果指定路径是目录而非文件的话，就会抛出 `IsADirectoryError` 异常。删除目录应该使用 `os.rmdir()` 函数。

同样的，对应于 `os.makedirs()` ，删除路径操作 `os.rmdir()` 也有一个递归删除的函数 `os.removedirs()` ，该函数会尝试从最下级目录开始，逐级删除指定的路径，几乎就是一个 `os.makedirs()` 的逆过程；一旦遇到非空目录即停止。

### 2.7 os.rename()

该函数的作用是将文件或路径重命名，一般调用格式为 `os.rename(src, dst)` ，即将 `src` 指向的文件或路径重命名为 `dst` 指定的名称。

注意，如果指定的目标路径在其他目录下，该函数还可实现文件或路径的“剪切并粘贴”功能。但无论直接原地重命名还是“剪切粘贴”，中间路径都必须要存在，否则就会抛出 `FileNotFoundError` 异常。如果目标路径已存在，Windows 下会抛出 `FileExistsError` 异常；Linux 下，如果目标路径为空且用户权限允许，则会静默覆盖原路径，否则抛出 `OSError` 异常，

和上两个函数一样，该函数也有对应的递归版本 `os.renames()` ，能够创建缺失的中间路径。

注意，这两种情况下，如果函数执行成功，都会调用 `os.removedir()` 函数来递归删除源路径的最下级目录。

### 2.8 os.getcwd()

“getcwd”实际上是“get the current working directory”的简写，顾名思义，也就是说这个函数的作用是“获取当前工作路径”。在程序运行的过程中，无论物理上程序在实际存储空间的什么地方，“当前工作路径”即可认为是程序所在路径；与之相关的“相对路径”、“同目录下模块导入”等相关的操作均以“当前工作路径”为准。

在交互式环境中，返回的就是交互终端打开的位置；而在 Python 文件中，返回的则是文件所在的位置。

在 Windows 下会有如下输出：

```
>>> os.getcwd()
'd:\\justdopython\\just\\do\\python'
```

Linux 下的输出则是：

```
>>> os.getcwd()
'/home/justdopython/just/do/python'
```

“chdir”其实是“change the directory”的简写，因此 `os.chdir()` 的用处实际上是切换当前工作路径为指定路径。其中“指定路径”需要作为参数传入函数 `os.chdir()` ，该参数既可以是文本或字节型字符串，也可以是一个文件描述符，还可以是一个广义的类路径（path-like）对象。若指定路径不存在，则会抛出 `FileNotFoundError` 异常。

在 Windows 下，调用该函数的效果为：

```
>>> os.chdir("d:/justdopython/just/do")
>>> os.getcwd()
'd:\\justdopython\\just\\do'
```

在 Linux 下的效果则是：

```
>>> os.chdir("/home/justdopython/just/do") # 也可将参数指定为“..”，即可切换到父目录
>>> os.getcwd()
'/home/justdopython/just/do'
```

有了这个函数，跨目录读写文件和调用模块就会变得非常方便了，很多时候也就不必再反复将同一个文件在各个目录之间复制粘贴运行，脚本完全可以坐镇中军，在一个目录下完成对其他目录文件的操作，正所谓“运筹帷幄之中，决胜于千里之外”也。

举例来说，可以通过将“当前工作目录”切换到父目录，从而直接访问父目录的文件内容：

```
>>> os.chdir("..")
>>> os.getcwd()
'D:\\justdopython\\just'
>>> with open("hello_justdopython.txt", encoding="utf-8") as f:
...     f.read()
...
'欢迎访问 justdopython.com，一起学习 Python 技术~'
>>> os.listdir()
['hello_justdopython.txt']
```

## 3. os.path 模块

其实这个模块是 `os` 模块根据系统类型从另一个模块导入的，并非直接由 `os` 模块实现，比如 `os.name` 值为 `nt` ，则在 `os` 模块中执行 `import ntpath as path` ；如果 `os.name` 值为 `posix` ，则导入 `posixpath` 。

使用该模块要注意一个很重要的特性： `os.path` 中的函数基本上是纯粹的字符串操作。换句话说，传入该模块函数的参数甚至不需要是一个有效路径，该模块也不会试图访问这个路径，而仅仅是按照“路径”的通用格式对字符串进行处理。

更进一步地说， `os.path` 模块的功能我们都可以自己使用字符串操作手动实现，该模块的作用是让我们在实现相同功能的时候不必考虑具体的系统，尤其是不需要过多关注文件系统分隔符的问题。

### 3.1 os.path.join()

这是一个十分实用的函数，可以将多个传入路径组合为一个路径。实际上是将传入的几个字符串用系统的分隔符连接起来，组合成一个新的字符串，所以一般的用法是将第一个参数作为父目录，之后每一个参数即使下一级目录，从而组合成一个新的符合逻辑的路径。

但如果传入路径中存在一个“绝对路径”格式的字符串，且这个字符串不是函数的第一个参数，那么其他在这个参数之前的所有参数都会被丢弃，余下的参数再进行组合。更准确地说，只有最后一个“绝对路径”及其之后的参数才会体现在返回结果中。

```
>>> os.path.join("just", "do", "python", "dot", "com")
```

```
>>> os.path.join("just", "do", "d:/", "python", "dot", "com")
'd:/python\\dot\\com'
>>>
>>> os.path.join("just", "do", "d:/", "python", "dot", "g:/", "com")
'g:/com'
```

### 3.2 os.path.abspath()

将传入路径规范化，返回一个相应的绝对路径格式的字符串。

也就是说当传入路径符合“绝对路径”的格式时，该函数仅仅将路径分隔符替换为适应当前系统的字符，不做其他任何操作，并将结果返回。所谓“绝对路径的格式”，其实指的就是一个字母加冒号，之后跟分隔符和字符串序列的格式：

```
>>> os.path.abspath("a:/just/do/python")
'a:\\just\\do\\python'
>>> # 我的系统中并没有 a 盘
```

当指定的路径不符合上述格式时，该函数会自动获取当前工作路径，并使用 `os.path.join()` 函数将其与传入的参数组合成为一个新的路径字符串。示例如下：

```
>>> os.path.abspath("ityouknow")
'D:\\justdopython\\ityouknow'
```

### 3.3 os.path.basename()

该函数返回传入路径的“基名”，即传入路径的最下级目录。

```
>>> os.path.basename("/ityouknow/justdopython/IAmBasename")
'IAmBasename'
>>> # 我的系统中同样没有这么一个路径。可见 os.path.basename() 页是单纯进行字符串处理
```

整这个函数要注意的一点是，返回的“基名”实际上是传入路径最后一个分隔符之后的子字符串，也就是说，如果最下级目录之后还有一个分隔符，得到的就会是一个空字符串：

```
>>> os.path.basename("/ityouknow/justdopython/IAmBasename/")
''
```

### 3.4 os.path.dirname()

与上一个函数正好相反，返回的是最后一个分隔符前的整个字符串：

```
>>> os.path.dirname("/ityouknow/justdopython/IAmBasename")
'/ityouknow/justdopython'
>>>
>>> os.path.dirname("/ityouknow/justdopython/IAmBasename/")
'/ityouknow/justdopython/IAmBasename'
```

### 3.5 os.path.split()

哈哈实际上前两个函数都是弟弟，这个函数才是老大。

函数 `os.path.split()` 的功能就是将传入路径以最后一个分隔符为界，分成两个字符串，并打包成元组的形式返回；前两个函数 `os.path.dirname()` 和 `os.path.basename()` 的返回值分别是函数 `os.path.split()` 返回值的第一个、第二个元素。就连二者的具体实现都十分真实：

```
def basename(p):
```

```
def dirname(p):
    """Returns the directory component of a pathname"""
    return split(p)[0]
```

通过 `os.path.join()` 函数又可以把它们组合起来得到原先的路径。

### 3.6 os.path.exists()

这个函数用于判断路径所指向的位置是否存在。若存在则返回 `True` ，不存在则返回 `False` ：

```
>>> os.path.exists(".")
True
>>> os.path.exists("./just")
True
>>> os.path.exists("./Inexistence") # 不存在的路径
False
```

一般的用法是在需要持久化保存某些数据的场景，为避免重复创建某个文件，需要在写入前用该函数检测一下相应文件是否存在，若不存在则新建，若存在则在文件内容之后增加新的内容。

### 3.7 os.path.isabs()

该函数判断传入路径是否是绝对路径，若是则返回 `True` ，否则返回 `False` 。当然，仅仅是检测格式，同样不对其有效性进行任何核验：

```
>>> os.path.isabs("a:/justdopython")
True
```

### 3.8 os.path.isfile() 和 os.path.isdir()

这两个函数分别判断传入路径是否是文件或路径，注意，此处会核验路径的有效性，如果是无效路径将会持续返回 `False` 。

```
>>> # 无效路径
>>> os.path.isfile("a:/justdopython")
False
>>>
>>> # 有效路径
>>> os.path.isfile("./just/plain_txt")
True
>>>
>>> # 无效路径
>>> os.path.isdir("a:/justdopython/")
False
>>> # 有效路径
>>> os.path.isdir("./just/")
True
```

## 4. 总结

本文详细介绍了与操作系统交互的 `os` 模块中一些常用的属性和函数，基本可以覆盖初阶的学习和使用。有了这些功能，我们已经可以写出一些比较实用的脚本了。

除了文中介绍的函数外， `os` 模块还有很多更加复杂的功能，但大多是我们暂时用不到的，以后用到会进一步讲解。

> 示例代码：python-100-days

## 5. 参考资料

Python3 文档-标准库

Python3 文档-库-os

发布于 2020-06-26 17:11

Python 文档 Linux 操作系统

写下你的评论...

**14 条评论** 默认 最新

**金色旭光**
特地登录给你你点赞，这一份文档是高质量的，不仅写了os模块，还写出优缺点，以及衍生功能的最佳实践，可以说是很有思想的。
2022-11-11 回复 15

**莫辞奕**
2.4代码的第6行，是否应该改成 if(os.path.isfile(file_dir + "\\" + directory)): 前面如果不加上父目录的话，我的环境里运行结果不对。
2023-01-10 回复 4

**柠檬陈**
Linux的加上'/'，os.path.isfile()参数是路径
2023-08-24 回复 喜欢

**S Q**
感谢大佬分享。
03-23 回复 喜欢

**知乎用户**
感谢大佬
03-23 回复 喜欢

**青青草原一头狼**
感谢分享
2023-08-01 回复 喜欢

**知乎用户9vF5e2**
👍
2023-03-28 回复 喜欢

**中二的Banana**
纠正一下：os.path.isdir()用于判断对象是否为一个目录 而不包含文件
2023-03-27 回复 喜欢

**卡片card**
我刚刚试了，对象如果是一个包含文件的目录返回的也是True，你可以试试
2023-04-24 回复 喜欢

**陈醋**
感谢 很详细
2021-10-20 回复 喜欢

**武小武**
感谢
2021-05-12 回复 喜欢

2022-09-24 回复 喜欢

点击查看全部评论 ›

写下你的评论...

**好好学习**

**推荐阅读**

**Python文件操作，OS模块**

编码声明： # coding = 'utf-8' # coding:'utf-8' # _*_ coding:<encoding name> _*_ python的文件对象：文件系统和文件： 文件系统时Os用于明确...

sharon

**Python3的os模块**

一、什么是os模块os模块提供了多数操作系统的功能接口函数。当os模块被导入后，它会自适应于不同的操作系统平台，根据不同的平台进行相应的操作，在python编程时，经常和文件、目录打交道，...

受限玻尔兹曼鸡

**Python 做 UI 超 easy！(5)——继续改进，使用 Combo...**

江川一尾 发表于一起 Py...

**Python 打包工具对比，Nuitka vs Pyinstaller**

py2exe，Pyinstaller，Cx_freeze，Nuitka都可以完成python打包的任务。 Pyinstaller和Nuitka都号称跨平台，但其实顶多只能算是工具本身跨平台，实际体验中不仅打包产生的文件不能跨...

知乎用户QhT0FW